IASYS Roadwork Challenge 2026

Student assignment

Master in Critical Computing Systems Engineering | Ricardo Severino

1 Mission and scope

Develop a C++17 ROS2 system that drives a simulated vehicle safely through a roadwork zone. Interpret a visual STOP or GO instruction, assess obstacles and opposing traffic, plan a feasible bypass, control the vehicle and stop at the destination.

Implement the complete sense–plan–act chain in at least three ROS2 nodes. This requirement applies to both deliveries: D1 covers S1–S2 and contributes 30%; D2 extends the same system to S1–S5 and contributes 70%.

Provided resources and student work

The Student v2 workspace provides the simulator, embedded evaluator, external message contracts, RViz configuration, launch files and minimal C++ node entry points. The starter intentionally remains stationary. It does not provide sensor subscriptions, internal message definitions, command publication or solution algorithms. There is no ros_runner.hpp.

Use Ubuntu 24.04 and ROS2 Jazzy. No Autoware stack, Gazebo, CARLA, GPU or neural-network training is required. Localisation is provided; autonomous behaviour and its ROS integration are your responsibility.

Required architecture

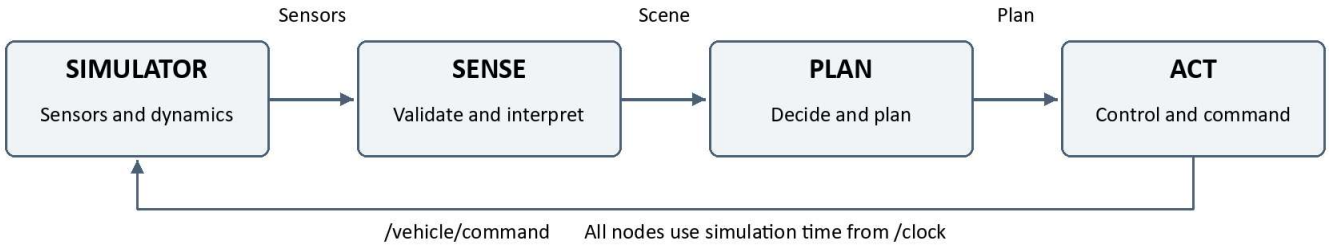


Figure 1 Minimum three-node architecture. Scene and Plan denote student-designed ROS interfaces.

Stage	Student responsibility
Sense	Subscribe to external observations; validate and interpret sensor data; publish perception evidence.
Plan	Subscribe to perception results; implement decisions, path planning and the speed policy; publish a plan.
Act	Subscribe to the plan; implement control and validity checks; publish /vehicle/command.

Three functions inside one node do not satisfy this requirement. Nodes must exchange meaningful ROS messages through observable topics. You may add nodes or distribute tracking between Plan and Act, provided the responsibilities and interfaces are documented. Only one node may publish vehicle commands.

2 ROS interfaces and sensor information

Use metres, radians and seconds of simulation time. The world frame is map; the vehicle frame is base_link. The x axis follows the road and positive y points left. Students define and implement their own internal interface package without changing the supplied external message contracts.

Topic	Message type	Rate
/vehicle/odometry	nav_msgs/msg/Odometry	20 Hz
/lidar/scan	sensor_msgs/msg/LaserScan	10 Hz
/radar/detections	roadwork_interfaces/msg/RadarDetections	10 Hz
/camera/image_raw	sensor_msgs/msg/Image	10 Hz
/infra/roadwork/beacon	roadwork_interfaces/msg/RoadworkBeacon	2 Hz[1]
/vehicle/command	roadwork_interfaces/msg/VehicleCommand	20 Hz[2]
/clock	rosgraph_msgs/msg/Clock	20 Hz

[1] Except during the S4 outage. [2] Expected command publication rate.

Odometry, LiDAR, radar and camera use best-effort SensorDataQoS. Beacon and command topics use reliable, volatile QoS with depth 10. A reliable-only sensor subscriber may not match a best-effort publisher. Enable use_sim_time=true in every solution node and derive timestamps from /clock.

What each sensor provides

LiDAR: 360 co-located rays, from −π inclusive to π exclusive, in one-degree increments. Range is 0.10–40 m; positive infinity means no return. Bounded range noise is controlled by noise. Project observations consistently into the frame used by your planner.

Radar: detections contain range, azimuth and radial velocity, within 80 m and ±60 degrees. There are no object IDs or ground-truth classes. Negative radial velocity means closing and includes ego motion: a static obstacle may therefore appear to approach. Range noise is ±3 × noise, velocity noise ±noise m/s and angle noise ±0.002 rad.

Camera: a 96 × 64 packed RGB8 crop of an infrastructure panel, with frame panel_crop. Red means STOP, green GO, and blank or ambiguous evidence UNKNOWN. This is not a perspective driving image. No semantic class, CameraInfo or confidence value is supplied.

Beacon: timestamped work-zone geometry, entry and exit, speed limit and pass_side=+1 for passing on the left. Its validity interval is 1.2 s. Geometry may be cached under a documented policy; cached geometry is not fresh permission to enter.

Permitted information

Algorithms may use the five observation topics and /clock, together with the published road geometry, vehicle limits and mission goal. **Work-zone geometry must come from the beacon, permission from images and obstacle evidence from sensors.**

Do not use /world/markers, /challenge/metrics, scenario IDs, random seeds, event schedules, simulator object lists or internal state to select actions. Debug visualisation and evaluator data are for inspection and assessment only. The source boundary is enforced through code review.

3 Vehicle model and common scenarios

Quantity	Contract
Road and start	Road edges y = ±3.5 m; own-lane centre y = −1.75 m. Start at (5, −1.75), heading zero, at rest.
Vehicle	Kinematic bicycle; wheelbase 2.4 m. Collision disc radius 0.90 m, centred on the pose reference.
Motion limits	Speed limit 4 m/s; acceleration ≤1.5 m/s²; braking ≤3 m/s²; steering ±0.45 rad; steering slew ≤0.6 rad/s.
Timing	Fixed simulation step 0.05 s. An episode times out after 180 s of simulation time.
Goal	Aim near (80.2, −1.75). Completion requires x ≥80 m, |y + 1.75| <0.6 m, |heading| <0.15 rad and speed <0.15 m/s.

Five separate scenarios

ID	Event	Required response
S1	GO throughout, with static roadworks.	Pass left, return to the own lane and stop at the goal.
S2	STOP until t = 10 s, then GO.	Stop before the vehicle envelope crosses the entry line; resume on valid evidence.
S3	GO with an approaching vehicle in the opposing lane.	Use sensor evidence to yield and establish a safe opportunity to traverse.
S4	Blank camera and no new beacon during 2 ≤ t <8 s.	Recognise unknown or stale evidence; wait before entry and recover after valid data returns.
S5	An obstacle appears during traversal and remains for 8 s.	Detect it, respond within braking limits and complete after the corridor becomes safe.

The entry line is x = 25 m. The nominal work region is x = 42–49 m; two radius-1 m obstacles are centred at (43 + work_shift, −1.75) and (47 + work_shift, −1.75). The exit is x = 68 + work_shift.

In S3, a radius-0.90 m vehicle starts at (82, 1.75) and travels towards decreasing x at traffic_speed. In S5, the first ego position x ≥45 + work_shift triggers a radius-0.65 m obstacle at (57 + work_shift, 1.75). The trigger is designed to leave stopping margin; you must account for sensing, processing and command delays.

Assessment variations

Permitted ranges are work_shift from −2 to +2 m, traffic_speed from 2 to 4 m/s and noise from 0 to 0.05 m. Seeds vary measurement noise. Every group uses the same assessment combinations. Selected combinations may be withheld, but no new scenario class or out-of-range disturbance will be introduced. Event schedules support reproducibility; do not hard-code timed manoeuvres.

4 Required behaviour and safety rules

Perception and temporal validity

Classify the panel as STOP, GO or UNKNOWN and document a confidence or validity rule. Use temporal evidence to avoid unstable decisions. Process LiDAR for obstacles or free space and radar for moving traffic, accounting for coordinate transformations and ego motion.

Preserve acquisition timestamps. Check message availability, age, future timestamps and malformed values, including at internal interfaces. A missing message is not an empty scene; repeated old images must not accumulate fresh GO evidence. Explain how measurements captured at different times are associated and what error this introduces.

Decision planning and control

Implement an explicit decision model with observable transitions. Require fresh authorisation and sufficient obstacle evidence before entry. GO authorises consideration of a manoeuvre; it does not override approaching traffic or an obstacle.

Define the commitment point and the response to information loss before and after commitment. Cached beacon geometry may support an ongoing manoeuvre, but obstacle decisions must continue to use current evidence. Document how the system recovers and how a new episode resets its state.

Generate or select a feasible path that passes the works on the left and returns to the original lane. Track it in closed loop with bounded commands. Explain the speed policy, stopping distances and delay margins. Publish perception outputs, decision state and planned path, and record enough information to diagnose failures.

Evaluator rules

- No contact with obstacle discs, road-boundary crossing or speed violation. Speed above 4.05 m/s is a violation; the 0.05 m/s tolerance is numerical, not an additional operating allowance.
- The vehicle front, represented by $x + 0.90$ m, must not cross $x = 25$ m during STOP or the S4 blackout. Entry, lane and speed violations remain latched for the episode.
- Opposing-lane occupancy occurs when $y + 0.90\ \text{m} > 0$. It is prohibited entirely before the entry region, when $x + 0.90\ \text{m} < 25$, and entirely beyond the exit region, when $x - 0.90\ \text{m} > \text{exit_x}$.
- A mission passes only if the goal conditions are met within 180 s and no collision, off-road, entry, lane or speed violation has occurred. Collision or leaving the road terminates the episode immediately.

Nonfinite commands, future timestamps or commands older than 0.30 s trigger simulator braking while holding steering. This watchdog does not provide obstacle avoidance or replace your safety strategy. The actuator accepts speed targets up to 8 m/s, but the challenge speed limit remains 4 m/s.

Model limitations

Odometry is exact. Collision geometry is circular. LiDAR is two-dimensional; radar does not model occlusion or multipath; the image is a panel crop. Discuss these limitations explicitly. Successful simulated runs do not establish real-world vehicle safety.

5 Deliveries and assessment

Work in groups of three. A fourth member requires prior agreement and a clear allocation of responsibilities. Deadlines will be announced on Moodle. Both deliveries must implement the complete chain in at least three ROS2 nodes.

D1 Intermediate delivery worth 30 percent

Solve S1 and S2 with one general policy. Submit executable source and build instructions; an architecture diagram; internal interface definitions; the decision model; six results from two scenarios × three seeds; a report of up to four pages; and a video of up to three minutes. Include an initial failure analysis and remaining risks.

S3–S5 behaviour is not assessed at D1. Freshness handling, safe entry, LiDAR clearance, control and final stopping remain part of S1/S2. Preserve the exact D1 submission for comparison.

D2 Final delivery worth 70 percent

Extend the same implementation to all five scenarios. Submit complete source, reproducible instructions, logs and configurations; at least 15 default runs from five scenarios × three seeds; at least two additional geometry or traffic variations; a baseline or ablation comparison; an English technical report of up to eight pages; and a video of up to five minutes.

Explain changes after D1 feedback, failures, limitations, each member's contribution and all reuse. Radar-based yielding, information-loss recovery and S3–S5 obstacle responses are assessed at D2.

Reporting and individual defence

Use the course LNCS report format. Page limits include figures and references. Report unsuccessful runs as well as successes. Demonstration and individual discussion are mandatory. Declare reused code, libraries and AI-assisted work; every member must be able to explain and modify the submitted implementation.

6 Running the challenge and recording evidence

Build and launch the Student workspace

Extract IASYS_Roadwork_Student_v2.zip, open a terminal in the extracted workspace root and source only ROS2 Jazzy. The workspace root is the folder containing src and scripts.

```
source /opt/ros/jazzy/setup.bash
bash scripts/check_environment.sh
rosdep install --from-paths src --ignore-src -r -y --rosdistro jazzy
colcon build --symlink-install
source install/setup.bash
ros2 launch roadwork_sim challenge.launch.py scenario:=1
```

The empty starter launches three nodes and keeps the vehicle stationary because it publishes no commands. Confirm the road, scan and panel display in RViz before implementing the pipeline. Use Ctrl+C to stop; each episode needs a fresh launch.

```
# Example run without visualisation and with a unique output file
ros2 launch roadwork_sim challenge.launch.py scenario:=5 seed:=42 \
  rviz:=false visualize:=false result_path:=/tmp/team_s5_seed42.json
```

To run your own packages separately, add student:=false to disable the starter nodes. Use one command publisher and an isolated ROS_DOMAIN_ID per group on a shared network. Never run two simulator episodes in the same domain.

Experimental evidence

For each run, record scenario, seed, work_shift, traffic_speed, noise, software version, hardware and whether visualisation was enabled. Use a distinct result_path. The evaluator writes terminal JSON and also publishes /challenge/metrics; completed episodes retain the final state until stopped.

Report completion, every violation, minimum clearance, maximum speed, elapsed simulation time and stopped time. The rejected_commands count includes startup before the first valid command; distinguish startup from failures during motion.

For D2, also measure recognition error on held-out or independently generated image samples, reaction latency and tracking error against your own planned path. Define the measurements and compare the final method with a simpler baseline or an ablated version. These measurements are not all provided by the evaluator.

Before submission

- Rebuild from a clean workspace and ensure another person can reproduce the results from your README.
- Verify at least three ROS2 nodes, meaningful intermediate topics, documented units and QoS, and exactly one command publisher.
- Test stale or missing inputs and loss of each pipeline node. Explain the stopping and recovery behaviour.
- Include source, interface definitions, launch/configuration files, tests, logs, results, reports and videos; document all dependencies and contributions.

Consult docs/INTERFACES.md for the full external contract and docs/SCENARIOS.md for exact event definitions. The student-designed architecture and internal interfaces must be documented in your submission.